boba_extension_dim04

Dimension 04: Cross-Browser Compatibility Matrix

Comprehensive Cross-Browser Compatibility & Polyfill Strategy for Torrent Extraction Extension

Research Date: July 2025 **Author**: Technical Research Team
Scope: Chrome, Firefox, Opera, Yandex Browser, Chromium (generic), Edge

---

Table of Contents

---

1. Executive Summary

This document provides a comprehensive compatibility matrix and polyfill strategy for developing and distributing a torrent extraction extension across Chrome, Firefox, Opera, Yandex Browser, Chromium, and Microsoft Edge. Key findings:

Finding	Impact
Chrome mandates MV3 (MV2 deprecated June 2025)	Must use service workers, declarative APIs
Firefox supports MV2 + MV3	No plans to deprecate MV2; webRequest blocking still works in MV3
Opera is Chromium-based	Uses `chrome.*` namespace, supports CRX files directly
Yandex Browser supports Chrome extensions	Install via `browser://tune/` or Chrome Web Store
Edge is Chromium-based (v79+)	Uses `chrome.*` namespace, MV3 supported
webextension-polyfill bridges API gaps	Enables Promise-based `browser.*` API on all Chromium browsers
Build tools (WXT, vite-plugin-web-extension)	Automate multi-browser manifest generation

2. Browser Overview

2.1 Browser Identification

Browser	Engine	Extension API	Manifest Support	Extension Store
Chrome	Blink (Chromium)	`chrome.*`	MV3 only (MV2 deprecated)	Chrome Web Store
Firefox	Gecko	`browser.*` + `chrome.*`	MV2 + MV3	addons.mozilla.org (AMO)
Opera	Blink (Chromium)	`chrome.*`	MV2 + MV3	Opera Add-ons
Yandex Browser	Blink (Chromium)	`chrome.*`	MV3 (Chrome-compatible)	Chrome Web Store / `browser://tune/`
Chromium (generic)		`chrome.*`	MV2 + MV3	No store (manual install)

Browser	Engine	Extension API	Manifest Support	Extension Store
	Blink (open-source)			
Edge	Blink (Chromium v79+)	`chrome.*`	MV2 + MV3	Edge Add-ons

2.2 Browser-Specific Identifiers (User Agent)

```
// Browser detection via user agent string
// NOTE: Use feature detection as primary method; UA sniffing as
        last resort

const ua = navigator.userAgent;

// Detection rules (order matters)
const isFirefox = ua.includes('Firefox') && !
        ua.includes('Seamonkey');
const isOpera   = ua.includes('OPR/') || ua.includes('Opera');
const isEdge    = ua.includes('Edg/');  // "Edg/" not "Edge"
const isChrome  = ua.includes('Chrome') && !isOpera && !isEdge;
const isYandex  = ua.includes('YaBrowser') ||
        ua.includes('Yowser');
const isChromium = ua.includes('Chromium') && !isChrome;
```

Claim: Opera 15+ uses “OPR/” in its user agent string, not “Opera” except for the legacy Presto-based Opera 12 and earlier. Source: MDN Web Docs - Browser detection using the user agent URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Browser_detection_using_the_user_agent Date: 2026-01-06 Excerpt: “|Opera 15+ (Blink-based engine)|`OPR/xyz`||” Context: UA sniffing for modern Opera versions Confidence: high

---

3. Compatibility Matrix

3.1 Feature Compatibility Matrix

Feature	Chrome	Firefox	Opera
Manifest V2	DEPRECATED	SUPPORTED	SUPPORTED
Manifest V3	SUPPORTED	SUPPORTED	SUPPORTED
`chrome.*` namespace	Native	Supported	Native
`browser.*` namespace	No	Native	No
Service Worker (MV3)	Required	Not supported	Required

Feature	Chrome	Firefox	Opera
Event Page/ Background Script	No (MV3)	Required	No (MV3)
`browser.webRequest` blocking	DNR only	SUPPORTED (MV3)	DNR only
`declarativeNetRequest`	SUPPORTED	SUPPORTED	SUPPORTED
`browser.storage.sync`	SUPPORTED	SUPPORTED	No
`browser.storage.managed`	SUPPORTED	SUPPORTED	No
Promise-based APIs	Partial (MV3)	Full	Via polyfill
Callback-based APIs	Full	Full	Full
Offscreen Documents	SUPPORTED	No	SUPPORTED
`browser.scripting` API	SUPPORTED	Partial	SUPPORTED
Content Script Injection	`scripting` API	Both	`scripting` API
ES Module Background	SUPPORTED	SUPPORTED	SUPPORTED
Native Messaging	SUPPORTED	SUPPORTED	SUPPORTED
`sidebarAction` API	No	SUPPORTED	SUPPORTED
`identity` API	Full	Partial	`launchWebAuthFlow` only
`commands` API	SUPPORTED	SUPPORTED	+ `_execute_sidebar_action`

3.2 Manifest Key Compatibility Matrix

Manifest Key	Chrome	Firefox	Opera	Yand
`manifest_version`	3	2 or 3	2 or 3	3
`background.service_worker`	REQUIRED (MV3)	Ignored (uses `scripts`)	REQUIRED (MV3)	REQ (MV3
`background.scripts`	MV2 only	SUPPORTED (MV3)	MV2 only	MV2
`background.page`	MV2 only	SUPPORTED	MV2 only	MV2
`action` (MV3)	SUPPORTED	SUPPORTED	SUPPORTED	SUPP
`browser_action` (MV2)	DEPRECATED	SUPPORTED	SUPPORTED	SUPP
`browser_specific_settings`	Ignored	REQUIRED (MV3 ID)	Ignored	Ignor
`content_security_policy`	MV3 format	Both	MV3 format	MV3
`web_accessible_resources`	MV3 format	Partial (no `use_dynamic_url`)	MV3 format	MV3
`host_permissions`	SUPPORTED	SUPPORTED	SUPPORTED	SUPP
`optional_host_permissions`	SUPPORTED	SUPPORTED	SUPPORTED	SUPP

Manifest Key	Chrome	Firefox	Opera	Yand
`permissions`	SUPPORTED	SUPPORTED	SUPPORTED	SUPP
`optional_permissions`	SUPPORTED	SUPPORTED	SUPPORTED	SUPP
`incognito`	`spanning`/ `split`/ `not_allowed`	`spanning` only	SUPPORTED	SUPP

3.3 Critical Differences for Torrent Extension

For a torrent extraction extension specifically, these differences matter most:

Feature	Chrome/Opera/ Yandex/Edge	Firefox	Strategy
HTTP/ Fetch from content script	Requires CORS or background	Same	Use background service worker for all fetch operations
URL pattern matching	`*://*.torrentsite.com/*`	Same	Standard match patterns work everywhere
Magnet link handling	`webNavigation` + `tabs`	Same	Use `tabs.onUpdated` to detect `magnet:` URLs
File download API	`chrome.downloads`	`browser.downloads`	Use polyfill
Clipboard write	`clipboardWrite` permission	Same	Same permission
Storage for torrent history	`chrome.storage.local`	`browser.storage.local`	Use polyfill
Context menus	`chrome.contextMenus`	`browser.menus`	Use polyfill (maps `menus` to `contextMenus`)

4. Manifest Differences Per Browser

4.1 Cross-Browser Manifest Template

```
{
  "{{chrome}}.manifest_version": 3,
  "{{firefox}}.manifest_version": 3,
  "{{opera}}.manifest_version": 3,
  "{{edge}}.manifest_version": 3,
  "name": "Torrent Extractor",
  "version": "1.0.0",
  "description": "Extract and manage torrent magnet links",
  "icons": {
    "16": "icons/icon-16.png",
    "32": "icons/icon-32.png",
    "48": "icons/icon-48.png",
    "128": "icons/icon-128.png"
  },
  "{{chrome}}.background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "{{firefox}}.background": {
    "scripts": ["background.js"],
    "type": "module"
  },
  "{{opera}}.background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "{{edge}}.background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "{{chrome}}.action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png"
    }
  },
  "{{firefox}}.action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png"
    },
    "browser_style": true
  },
  "{{opera}}.action": {
    "default_popup": "popup.html",
```

```
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png"
    }
  },
  "{{edge}}.action": {
    "default_popup": "popup.html",
    "default_icon": {
      "16": "icons/icon-16.png",
      "32": "icons/icon-32.png"
    }
  },
  "permissions": [
    "activeTab",
    "storage",
    "downloads",
    "contextMenus",
    "notifications",
    "scripting",
    "tabs"
  ],
  "{{chrome}}.host_permissions": [
    "http://*/",
    "https://*/"
  ],
  "{{firefox}}.host_permissions": [
    "http://*/",
    "https://*/"
  ],
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"],
      "run_at": "document_idle"
    }
  ],
  "{{firefox}}.browser_specific_settings": {
    "gecko": {
      "id": "torrent-extractor@yourdomain.com",
      "strict_min_version": "109.0"
    }
  },
  "web_accessible_resources": [
    {
      "resources": ["icons/*"],
      "matches": ["<all_urls>"]
    }
  ]
}
```

4.2 Browser-Specific Manifest Fields

Firefox: `browser_specific_settings.gecko`

```
{
  "browser_specific_settings": {
    "gecko": {
      "id": "extensionname@example.org",
      "strict_min_version": "109.0",
      "strict_max_version": "*",
      "update_url": "https://example.com/updates.json"
    },
    "gecko_android": {
      "strict_min_version": "121.0"
    }
  }
}
```

Claim: The extension ID is required for Manifest V3 in Firefox. AMO does not auto-assign an ID for MV3 extensions. Source: MDN Web Docs - browser_specific_settings URL: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/manifest.json/browser_specific_settings Date: 2026-04-20 Excerpt: “The extension ID. Optional for Manifest V2 (although setting an ID is recommended) and required for signing Manifest V3 extensions. If you don’t provide a value for Manifest V2 extensions, AMO assigns a GUID to the extension when it is signed.” Context: MV3 extensions MUST include browser_specific_settings.gecko.id Confidence: high

Opera: `minimum_opera_version` and `sidebar_action`

```
{
  "minimum_opera_version": "77",
  "sidebar_action": {
    "default_icon": "icons/sidebar-icon.png",
    "default_title": "Torrent Extractor",
    "default_panel": "sidebar.html"
  }
}
```

Edge: No specific fields required

Edge accepts standard Chrome-compatible manifests without modifications.

---

5. API Namespace Differences

5.1 The `chrome.*` vs `browser.*` Problem

Aspect	`chrome.*` (Chromium)	`browser.*` (Firefox)
API Style	Callback-based (MV2), Promise added (MV3)	Promise-based (native)
Error Handling	`chrome.runtime.lastError`	Promise rejection
Return Value	`undefined` (via callback)	`Promise`
Browser Support	Chrome, Opera, Edge, Yandex	Firefox only

5.2 Code Comparison

Callback style (chrome.*):

```
chrome.tabs.query({ active: true, currentWindow: true }, (tabs)
        => {
  if (chrome.runtime.lastError) {
    console.error(chrome.runtime.lastError);
    return;
  }
  console.log(tabs[0].url);
});
```

Promise style (browser.*):

```
browser.tabs.query({ active: true, currentWindow: true })
  .then((tabs) => {
    console.log(tabs[0].url);
  })
  .catch((error) => {
    console.error(error);
  });
```

Async/await style (works with polyfill):

```
async function getCurrentTab() {
  try {
    const tabs = await browser.tabs.query({ active: true,
        currentWindow: true });
    return tabs[0];
  } catch (error) {
    console.error(error);
    return null;
```

```
  }
}
```

5.3 API Equivalence Table

Chrome (`chrome.*`)	Firefox (`browser.*`)	Notes
`chrome.tabs`	`browser.tabs`	Equivalent
`chrome.storage`	`browser.storage`	Equivalent
`chrome.runtime`	`browser.runtime`	Equivalent
`chrome.downloads`	`browser.downloads`	Equivalent
`chrome.contextMenus`	`browser.menus`	`menus` is Firefox-specific superset
`chrome.notifications`	`browser.notifications`	Opera doesn’t support Image/List/Progress types
`chrome.webRequest`	`browser.webRequest`	Firefox supports blocking in MV3; Chromium uses DNR
`chrome.scripting`	`browser.scripting`	executeScript, insertCSS, removeCSS
`chrome.action`	`browser.action`	MV3 unified action (was `browserAction`/`pageAction`)
`chrome.declarativeNetRequest`	`browser.declarativeNetRequest`	Available in Firefox 113+
`chrome.identity`	`browser.identity`	Opera only supports `launchWebAuthFlow()`
`chrome.sidebarAction`	`browser.sidebarAction`	Opera + Firefox only
`chrome.sessions`	`browser.sessions`	Opera: sync not supported
`chrome.storage.sync`	`browser.storage.sync`	Opera does NOT support sync
`chrome.storage.managed`	`browser.storage.managed`	Opera does NOT support managed
`chrome.bookmarks`	`browser.bookmarks`	Opera adds `getRootByName` method
`chrome.i18n`	`browser.i18n`	

Chrome (`chrome.*`)	Firefox (`browser.*`)	Notes
		Opera doesn't support `getUILanguage()`

---

6. WebExtension Polyfill Setup

6.1 What is webextension-polyfill?

The `webextension-polyfill` library by Mozilla enables Promise-based WebExtension APIs (the `browser.*` namespace) to run on Chromium-based browsers (Chrome, Opera, Edge, Yandex) with minimal or no changes.

Claim: The polyfill is officially supported on Chrome and Firefox (as a NO-OP), and unofficially supported on Opera and Edge as Chrome-compatible targets. Source: GitHub - mozilla/webextension-polyfill URL: https://github.com/mozilla/webextension-polyfill Date: 2024-05-14 Excerpt: "|Chrome| Officially Supported (with automated tests)|; |Firefox|Officially Supported as a NO-OP|; |Opera / Edge (>=79.0.309)|Unofficially Supported as a Chrome-compatible target|" Context: Support matrix from the polyfill's README Confidence: high

6.2 Installation

```
npm install --save-dev webextension-polyfill
npm install --save-dev @types/webextension-polyfill  # For
        TypeScript
```

6.3 Setup in Manifest V3

Option A: Include in manifest (for background/content scripts):

```
{
  "background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["browser-polyfill.js", "content.js"],
      "run_at": "document_idle"
    }
```

```
  ]
}
```

Option B: Import as module (recommended with bundlers):

```
// In your source files, import the polyfill
import browser from 'webextension-polyfill';

// Use browser.* APIs everywhere - works on all browsers
async function fetchTorrentData(url) {
  try {
    const response = await fetch(url);
    const data = await response.text();
    // Store using the unified API
    await browser.storage.local.set({ lastTorrentData: data });
    return data;
  } catch (error) {
    console.error('Failed to fetch:', error);
    throw error;
  }
}
```

6.4 HTML Pages (Popup, Options)

```
<!DOCTYPE html>
<html>
<head>
  <script type="module" src="browser-polyfill.js"></script>
  <script type="module" src="popup.js"></script>
</head>
<body>
  <!-- Popup content -->
</body>
</html>
```

6.5 Dynamic Script Injection

```
// When injecting content scripts dynamically, include the
        polyfill first
await browser.scripting.executeScript({
  target: { tabId: tab.id },
  files: ['browser-polyfill.js', 'injected-content.js']
});
```

6.6 Known Limitations

Claim: The polyfill does NOT polyfill API methods that are missing on Chrome but natively provided on Firefox. The extension must do its own “runtime feature detection” for those cases. Source: GitHub - mozilla/webextension-polyfill URL: https://github.com/

mozilla/webextension-polyfill Date: 2024-05-14 Excerpt: “This library doesn’t (and it is not going to) polyfill API methods or options that are missing on Chrome but natively provided on Firefox, and so the extension has to do its own ‘runtime feature detection’ in those cases.” Context: Scope limitation of the polyfill Confidence: high

Critical Limitations:

1. **No callback support**: Promise-based APIs do not support callbacks
2. **tabs.executeScript**: On Chrome, only immediate values are supported as return values (not Promises)
3. **Missing APIs not polyfilled**: Firefox-only APIs (e.g., `browser.menus`, `browser.sidebarAction`) are not added to Chrome
4. **API metadata dependency**: APIs not in `api-metadata.json` won’t be wrapped

6.7 Feature Detection Pattern

```
import browser from 'webextension-polyfill';

// Detect if a feature exists before using it
async function createContextMenu(items) {
  // Use browser.menus on Firefox, browser.contextMenus on Chrome
  const menuAPI = browser.menus || browser.contextMenus;

  if (!menuAPI) {
    console.warn('Context menus not supported');
    return;
  }

  for (const item of items) {
    await menuAPI.create(item);
  }
}

// Detect sidebar support
function isSidebarSupported() {
  return typeof browser.sidebarAction !== 'undefined';
}

// Detect webRequest blocking support (Firefox MV3)
function isWebRequestBlockingSupported() {
  return browser.webRequest &&
         browser.webRequest.onBeforeRequest &&
         typeof browser.webRequest.OnBeforeRequestOptions !==
        'undefined' &&
```

```
        browser.webRequest.OnBeforeRequestOptions?.includes('blocking');
}
```

---

7. Runtime Browser Detection

7.1 Method 1: runtime.getBrowserInfo() (Firefox only)

```
// This only works in Firefox
async function detectFirefox() {
  try {
    if (typeof browser !== 'undefined' && browser.runtime &&
        browser.runtime.getBrowserInfo) {
      const info = await browser.runtime.getBrowserInfo();
      // info.name = "Firefox", info.vendor = "Mozilla",
        info.version = "121.0"
      return { name: info.name, version: info.version, vendor:
        info.vendor };
    }
  } catch (e) {
    // Not Firefox
  }
  return null;
}
```

7.2 Method 2: runtime.getPlatformInfo() (All browsers)

```
// Returns platform info (OS, architecture) - same across browsers
async function getPlatformInfo() {
  const info = await browser.runtime.getPlatformInfo();
  // info.os: "mac", "win", "android", "cros", "linux", "openbsd"
  // info.arch: "arm", "x86-32", "x86-64", "mips", "mips64"
  return info;
}
```

7.3 Method 3: User Agent Detection (Last Resort)

```
function detectBrowser() {
  const ua = navigator.userAgent;

  // Order matters: check more specific first
  if (ua.includes('YaBrowser') || ua.includes('Yowser')) {
    return { name: 'yandex', engine: 'blink', chromium: true };
  }
  if (ua.includes('OPR/') || ua.includes('Opera')) {
    return { name: 'opera', engine: 'blink', chromium: true };
```

```
  }
  if (ua.includes('Edg/')) {
    return { name: 'edge', engine: 'blink', chromium: true };
  }
  if (ua.includes('Firefox') && !ua.includes('Seamonkey')) {
    return { name: 'firefox', engine: 'gecko', chromium: false };
  }
  if (ua.includes('Chrome') && !ua.includes('Chromium')) {
    return { name: 'chrome', engine: 'blink', chromium: true };
  }
  if (ua.includes('Chromium')) {
    return { name: 'chromium', engine: 'blink', chromium: true };
  }
  if (ua.includes('Safari') && !ua.includes('Chrome')) {
    return { name: 'safari', engine: 'webkit', chromium: false };
  }

  return { name: 'unknown', engine: 'unknown', chromium: false };
}
```

7.4 Method 4: Feature Detection (Recommended)

```
function detectBrowserByFeatures() {
  const hasBrowserAPI = typeof browser !== 'undefined';
  const hasChromeAPI = typeof chrome !== 'undefined';
  const hasRuntime = hasBrowserAPI && browser.runtime ||
        hasChromeAPI && chrome.runtime;

  // Firefox has both browser.* and chrome.* (chrome is alias)
  if (hasBrowserAPI && browser.runtime &&
        browser.runtime.getBrowserInfo) {
    return 'firefox';
  }

  // Chrome has chrome.* but NOT browser.* (unless polyfill
        loaded)
  if (hasChromeAPI && !hasBrowserAPI) {
    // Check if browser was defined by polyfill
    if (browser && browser._polyfill) {
      return 'chrome-with-polyfill';
    }
    return 'chrome';
  }

  // With polyfill, browser.* works everywhere
  // Use UA as fallback
  return detectBrowser().name;
}
```

7.5 Recommended Detection Strategy

```
// utils/browser-detection.js
import browser from 'webextension-polyfill';

export const BROWSERS = {
  CHROME: 'chrome',
  FIREFOX: 'firefox',
  OPERA: 'opera',
  EDGE: 'edge',
  YANDEX: 'yandex',
  CHROMIUM: 'chromium',
  UNKNOWN: 'unknown'
};

export async function getBrowserInfo() {
  // Try Firefox-specific API first
  if (browser.runtime.getBrowserInfo) {
    try {
      const info = await browser.runtime.getBrowserInfo();
      return {
        name: info.name.toLowerCase(),
        version: info.version,
        vendor: info.vendor,
        isChromium: false,
        isFirefox: true
      };
    } catch (e) {
      // ignore
    }
  }

  // Fallback to user agent
  const ua = navigator.userAgent;

  if (ua.includes('YaBrowser') || ua.includes('Yowser')) {
    return { name: BROWSERS.YANDEX, isChromium: true, isFirefox:
        false };
  }
  if (ua.includes('OPR/')) {
    return { name: BROWSERS.OPERA, isChromium: true, isFirefox:
        false };
  }
  if (ua.includes('Edg/')) {
    return { name: BROWSERS.EDGE, isChromium: true, isFirefox:
        false };
  }
  if (ua.includes('Chrome') && !ua.includes('Chromium')) {
    return { name: BROWSERS.CHROME, isChromium: true, isFirefox:
        false };
  }
  if (ua.includes('Chromium')) {
```

```
    return { name: BROWSERS.CHROMIUM, isChromium: true,
        isFirefox: false };
  }

  return { name: BROWSERS.UNKNOWN, isChromium: false, isFirefox:
        false };
}

export function isChromiumBased() {
  return typeof chrome !== 'undefined' &&
         chrome.runtime &&
         chrome.runtime.onInstalled &&
         typeof browser === 'undefined' ||
         (typeof browser !== 'undefined' && browser._polyfill);
}
```

---

8. Cross-Browser Build Pipeline

8.1 Option 1: WXT (Recommended)

WXT is the modern recommended framework for building cross-browser extensions. It handles manifests, builds, and dev mode automatically.

Claim: WXT supports all browsers, both MV2 and MV3, and provides dev mode with HMR and fast reload. It is frontend framework agnostic. Source: GitHub - wxt-dev/wxt URL: https://github.com/wxt-dev/wxt Date: 2023-06-25 Excerpt: “- Supports all browsers; - Supports both MV2 and MV3; - Dev mode with HMR & fast reload; - File based entrypoints” Context: WXT is a Next-gen Web Extension Framework built on Vite Confidence: high

```
# Initialize project
npx wxt@latest init

# Dev mode (auto-detects browser)
npm run dev

# Build for specific browsers
npx wxt build --browser chrome
npx wxt build --browser firefox
npx wxt build --browser edge
npx wxt build --browser opera

# Output structure
# .output/
#   chrome-mv3/
#   firefox-mv2/
#   edge-mv3/
```

wxt.config.ts:

```
import { defineConfig } from 'wxt';

export default defineConfig({
  manifest: {
    name: 'Torrent Extractor',
    permissions: ['activeTab', 'storage', 'downloads',
        'contextMenus', 'notifications', 'scripting', 'tabs'],
    host_permissions: ['http://*/', 'https://*/'],
  },
  // WXT auto-generates browser-specific manifests
});
```

8.2 Option 2: vite-plugin-web-extension

```
npm install --save-dev vite-plugin-web-extension
```

vite.config.ts:

```
import { defineConfig } from 'vite';
import webExtension from 'vite-plugin-web-extension';

const target = process.env.TARGET || 'chrome';

export default defineConfig({
  plugins: [
    webExtension({
      browser: target,
      manifest: target === 'chrome' ? 'manifest.chrome.json' :
        'manifest.firefox.json',
    }),
  ],
});
```

Manifest template with placeholders:

```
{
  "{{chrome}}.manifest_version": 3,
  "{{firefox}}.manifest_version": 3,
  "{{opera}}.manifest_version": 3,
  "{{edge}}.manifest_version": 3,
  "name": "Torrent Extractor",
  "version": "1.0.0",
  "{{chrome}}.background": {
    "service_worker": "background.js",
    "type": "module"
  },
  "{{firefox}}.background": {
    "scripts": ["background.js"],
    "type": "module"
```

```
  }
}
```

8.3 Option 3: Manual Build Scripts

package.json:

```
{
  "scripts": {
    "build:chrome": "TARGET=chrome vite build && npm run
        manifest:chrome",
    "build:firefox": "TARGET=firefox vite build && npm run
        manifest:firefox",
    "build:opera": "TARGET=opera vite build && npm run
        manifest:opera",
    "build:edge": "TARGET=edge vite build && npm run
        manifest:edge",
    "build:all":
        "npm run build:chrome && npm run build:firefox && npm run
        build:opera && npm run build:edge",
    "manifest:chrome": "cp manifests/chrome.json dist/chrome/
        manifest.json",
    "manifest:firefox": "cp manifests/firefox.json dist/firefox/
        manifest.json",
    "package:chrome": "cd dist/chrome && zip -r ../../packages/
        torrent-extractor-chrome.zip .",
    "package:firefox": "cd dist/firefox && zip -r ../../packages/
        torrent-extractor-firefox.zip .",
    "package:all": "npm run package:chrome && npm run
        package:firefox && npm run package:opera && npm run
        package:edge"
  }
}
```

8.4 Build Output Structure

```
packages/
  torrent-extractor-chrome.zip      # Chrome Web Store package
  torrent-extractor-firefox.zip     # AMO package
  torrent-extractor-opera.zip       # Opera Add-ons package
  torrent-extractor-edge.zip        # Edge Add-ons package

dist/
  chrome/                           # Chrome unpacked
    manifest.json
    background.js
    content.js
    popup.html
    popup.js
    icons/
  firefox/                          # Firefox unpacked
    manifest.json
    background.js
```

```
  content.js
  popup.html
  popup.js
  icons/
edge/                              # Edge unpacked
opera/                             # Opera unpacked
```

---

9. Store Submission Checklist

9.1 Chrome Web Store

Requirement	Details	Status
Developer Account	Google account + $5 one-time fee	Required
Manifest Version	Must be MV3 (MV2 no longer accepted as of June 2025)	Required
Icon	128x128 PNG in ZIP; 96x96 actual icon + 16px transparent padding	Required
Screenshots	1280x800 (recommended) or 640x400; PNG or JPEG; min 1, max 5	Required (at least 1)
Small Promo Tile	440x280 PNG	Required
Large Promo Tile	1400x560 PNG (optional, for featured placement)	Optional
Description	Up to 16,000 characters	Required
Privacy Policy URL	Required if any data collection	Required if collecting data
Single Purpose	Brief explanation of what extension does	Required
Permission Justification	Explain why each permission is needed	Required
Remote Code		Required

Requirement	Details	Status
	Must answer “No” (no remotely hosted code)	
Review Time	Automatic, typically < 1 hour	
2FA	Required for developer account	Required
Registration Fee	$5 one-time	Required

Claim: Chrome Web Store charges a $5 one-time developer registration fee. This covers all future extensions (up to 20 per account). Source: Extension Radar URL: https://www.extensionradar.com/blog/chrome-web-store-developer-fee-2026 Date: 2026-02-16 Excerpt: “The fee is $5 USD. That’s it. - It is a one-time payment. - It covers all your future extensions. - It is valid for life.” Context: Cost comparison with other stores Confidence: high

9.2 Firefox AMO (addons.mozilla.org)

Requirement	Details	Status
Account	Firefox Account (free)	Required
Manifest Version	MV2 or MV3 (MV2 still supported)	Either
Extension ID	Required for MV3 (`browser_specific_settings.gecko.id`)	Required
Icon	Recommended: 48x48, 96x96, or 128x128 PNG	Recommended
Screenshots	1280x800 or 640x480 PNG; min 1, no max	Recommended
Description	Detailed description of functionality	Required
Privacy Policy	Required if transmitting any user data	Required if applicable
Source Code	May be required if using build tools or minification	Conditionally required
Data Collection Permissions	New requirement as of Nov 2025 for new extensions	Required
Review Process	Automatic (instant) + manual review after publication	
Registration Fee	Free	Free
		Available

Requirement	Details	Status
Unlisted Option	Can self-distribute without AMO listing	

Claim: Starting November 3rd, 2025, all newly submitted Firefox extensions must explicitly declare data collection practices via `browser_specific_settings.gecko.data_collection_permissions`. Source: GBHackers / Mozilla Add-ons Blog URL: https://gbhackers.com/mozilla-enforces-transparency-rules-new-firefox-extensions/ Date: 2025-10-29 Excerpt: “The compliance framework requires developers to specify data handling practices directly in the manifest.json file using the browser_specific_settings.gecko.data_collection_permissions key.” Context: New requirement for transparency in data collection Confidence: high

9.3 Opera Add-ons

Requirement	Details	Status
Account	Opera account (free)	Required
Manifest Version	MV2 or MV3	Either
Icon	128x128 (for store); 48x48 (for management page); 16x16 (favicon)	Required
Screenshots	612x408 preferred (max 800x600); white background; disable other extensions	Required
Description	Clear description of functionality	Required
No External JavaScript	All JS must be contained in extension	Required
Review Process	Manual (no SLA provided)	
Registration Fee	Free	Free
2FA	Not required	Not required

Claim: Opera Add-ons requires screenshots of 612x408 pixels as the preferred size, with a white background and other extensions disabled. Source: Opera Help - Publishing Guidelines URL: https://help.opera.com/en/extensions/publishing-guidelines/ Date: 2026-03-09 Excerpt: “Have a screenshot size of 612x408 pixels.

This is the preferable screenshot size. The maximum you go can go with it is 800x600 pixels. Take your screenshots with a white background.” Context: Opera-specific store requirements Confidence: high

9.4 Microsoft Edge Add-ons

Requirement	Details	Status
Account	Microsoft account (free)	Required
Manifest Version	MV2 or MV3	Either
Icon	300x300 recommended (min 128x128), 1:1 ratio	Required
Screenshots	640x480 or 1280x800 PNG; max 6	Optional
Small Promo Tile	440x280	Optional
Large Promo Tile	1400x560	Optional
Description	250-10,000 characters, thorough	Required
Privacy Policy	Required if collecting data	Conditionally required
Registration Fee	Free	Free
2FA	Required	Required
Review Process	Manual (no SLA)	

Claim: Microsoft Edge Add-ons store has specific requirements: 300x300 icon, 1280x800 screenshots, description 250-10,000 characters. Source: Microsoft Learn - Publish a Microsoft Edge extension URL: https://learn.microsoft.com/en-us/microsoft-edge/extensions/publish/publish-extension Date: 2026-05-05 Excerpt: “Extension logo: An image with an aspect ratio of 1:1 and a recommended size of 300 x 300 pixels. Screenshots: The size of the screenshots must be either 640 x 480 pixels or 1280 x 800 pixels.” Context: Edge store submission requirements Confidence: high

9.5 Yandex Browser

Yandex Browser does NOT have its own extension store. Extensions are distributed through:

Method	Process
Chrome Web Store	Install directly from CWS (extensions are compatible)
Developer Mode (`browser://tune/`)	Drag CRX3 file to `browser://tune/` page
Unpacked Loading	Enable developer mode at `browser://extensions/`, then “Load unpacked”

Claim: Yandex Browser supports extensions from Google Chrome and Opera catalogs. Custom extensions can be installed by dragging a CRX3 archive to browser://tune/. Source: Yandex Browser Help URL: https://browser.yandex.ru/help/en/personalization/extension Date: N/A (official docs) Excerpt: “You can install extensions from the third-party catalogs of Google Chrome and Opera, which are compatible with Yandex Browser… Archive the extension and save it as a CRX3 file. Go to browser://tune/ in the browser. Open the folder with the archive and drag the archive to the Yandex Browser window.” Context: Yandex Browser extension installation methods Confidence: high

9.6 Chromium (Generic / Open Source)

Chromium has no store. Extensions are loaded via:

Method	Process
Developer Mode	Go to `chrome://extensions/`, enable Developer Mode, click “Load unpacked”
Command Line	`--load-extension=/path/to/extension` (only in unbranded builds)
CRX File	Drag and drop CRX file to `chrome://extensions/`

10. Per-Browser Deep Dive

10.1 Chrome

Status: Chrome uses MV3 exclusively as of June 2025. MV2 extensions have been removed from the Chrome Web Store.

Key technical points: - Background scripts replaced by **service workers** (non-persistent, event-driven) - No DOM access in service workers (`window`, `document` unavailable) - `webRequest` blocking

replaced by `declarativeNetRequest` (DNR) - `chrome.*` namespace with callback-based APIs (MV2) or Promise support (MV3) - Offscreen documents for tasks requiring DOM access

```
// Chrome MV3 service worker example
// background.js

chrome.runtime.onInstalled.addListener(() => {
  console.log('Extension installed');
});

chrome.action.onClicked.addListener(async (tab) => {
  // Inject content script
  await chrome.scripting.executeScript({
    target: { tabId: tab.id },
    files: ['content.js']
  });
});

// Storage with Promise (MV3)
chrome.storage.local.set({ key: 'value' }).then(() => {
  console.log('Saved');
});
```

Claim: Google removed MV2 extensions from the Chrome Web Store in June 2025. Source: Wikipedia - Google Chrome URL: https://en.wikipedia.org/wiki/Google_Chrome Date: 2008-09-01 (updated) Excerpt: “Google removed the extensions using MV2 from their Chrome Web Store in June 2025.” Context: MV3 migration timeline Confidence: high

10.2 Firefox

Status: Firefox supports both MV2 and MV3. Mozilla has no plans to deprecate MV2.

Key technical points: - **MV3 uses event pages (background scripts)**, NOT service workers - `browser.webRequest` blocking STILL WORKS in MV3 (unlike Chrome) - `browser.*` namespace with native Promise support - Also supports `chrome.*` namespace for backward compatibility - Extension ID required in MV3 via `browser_specific_settings.gecko.id`

```
// Firefox MV3 background script (event page, NOT service worker)
// background.js

browser.runtime.onInstalled.addListener(() => {
  console.log('Extension installed');
});

// Firefox MV3 still supports blocking webRequest!
browser.webRequest.onBeforeRequest.addListener(
```

```
  (details) => {
    if (details.url.includes('tracker.example.com')) {
      return { cancel: true };
    }
  },
  { urls: ['<all_urls>'] },
  ['blocking']  // This works in Firefox MV3!
);
```

Claim: Firefox MV3 still allows blocking webRequest, unlike Chrome which replaced it with declarativeNetRequest. Source: dev.to - Firefox Extension Manifest V3 vs V2 URL: https://dev.to/weatherclockdash/firefox-extension-manifest-v3-vs-v2-what-actually-changed-5654 Date: 2026-05-04 Excerpt: “Firefox MV3 — still works! browser.webRequest.onBeforeRequest.addListener with [‘blocking’] - This matters for ad blockers and privacy tools - uBlock Origin works on Firefox MV3 because of this.” Context: Firefox MV3 webRequest blocking capability Confidence: high

Claim: Mozilla has no current plans to deprecate MV2. Source: Mozilla Add-ons Community Blog URL: https://blog.mozilla.org/addons/2024/05/14/manifest-v3-updates/ Date: 2024-05-14 Excerpt: “Mozilla has no current plans to deprecate MV2 as mentioned in our previous MV3 update.” Context: Mozilla’s position on MV2 deprecation Confidence: high

10.3 Opera

Status: Opera is Chromium-based and supports Chrome extensions with minimal modifications.

Key technical points: - Uses `chrome.*` namespace (identical to Chrome) - Supports CRX files directly - Has Opera-specific APIs: `opr.sidebarAction`, `opr.addons` - `minimum_opera_version` manifest field supported - Storage `sync()` and `managed()` NOT supported - Notifications: Image, List, Progress types not supported on Mac - Bookmarks API adds `getRootByName` method

```
// Opera-specific sidebar action (optional)
// Only works in Opera

if (typeof opr !== 'undefined' && opr.sidebarAction) {
  opr.sidebarAction.setPanel({
    panel: 'sidebar.html'
  });
}
```

10.4 Yandex Browser

Status: Yandex Browser is a Chromium fork that supports Chrome extensions.

Key technical points: - Based on Chromium (similar to Opera) - Supports `chrome.*` namespace - Extensions install from Chrome Web Store or Opera Store - Developer mode at `browser://extensions/` - Custom install via `browser://tune/` with CRX3 files - Some extensions may be blocked as "potentially dangerous" - Ignores extensions that change new tab appearance

```
// Yandex Browser is treated as Chrome for all API purposes
// No special code needed - just ensure Chrome compatibility
```

10.5 Edge

Status: Edge switched to Chromium (Blink) starting with version 79. Uses same extension model as Chrome.

Key technical points: - `chrome.*` namespace (fully compatible) - Supports MV2 and MV3 - Before v79: used `browser.*` namespace with EdgeHTML engine (now obsolete) - Edge Add-ons store accepts the same packages as Chrome Web Store - Additional `edge` key in `browser_specific_settings` (for Firefox compatibility)

```
// Edge is detected via Edg/ in user agent
// No code changes needed from Chrome

// Edge v79+ uses chrome.* APIs identically to Chrome
```

10.6 Chromium (Generic)

Status: Open-source Chromium builds have no store and require developer mode for extension loading.

Key technical points: - Same extension APIs as Chrome - `--load-extension` flag works in unbranded builds - No review process (no store) - Extensions auto-update must be managed manually or via `update_url` - No $5 developer fee (no store to publish to)

```
# Launch Chromium with extension loaded
chromium --load-extension=/path/to/extension \
         --enable-extensions \
         --remote-debugging-port=9222
```

---

11. Code Examples

11.1 Complete Cross-Browser Background Script

```
// background.js - Works on all browsers with webextension-
        polyfill
import browser from 'webextension-polyfill';

// Constants
const MAGNET_REGEX = /magnet:\?xt=urn:btih:[a-zA-Z0-9]*/gi;

// Initialize extension
browser.runtime.onInstalled.addListener((details) => {
  if (details.reason === 'install') {
    console.log('Torrent Extractor installed');
    setupContextMenus();
  }
});

// Setup context menus (works on all browsers)
async function setupContextMenus() {
  const menuAPI = browser.menus || browser.contextMenus;
  if (!menuAPI) return;

  await menuAPI.create({
    id: 'extract-torrents',
    title: 'Extract Torrent Links',
    contexts: ['page', 'link']
  });

  menuAPI.onClicked.addListener(handleContextMenuClick);
}

async function handleContextMenuClick(info, tab) {
  if (info.menuItemId === 'extract-torrents') {
    try {
      // Inject content script to extract magnet links
      const results = await browser.scripting.executeScript({
        target: { tabId: tab.id },
        func: extractMagnetLinks
      });

      const links = results[0]?.result || [];

      // Store extracted links
      await browser.storage.local.set({
        [`torrents_${tab.id}`]: {
          url: tab.url,
          title: tab.title,
          links: links,
          timestamp: Date.now()
```

```
        }
      });

      // Show notification
      await browser.notifications.create({
        type: 'basic',
        iconUrl: 'icons/icon-128.png',
        title: 'Torrents Extracted',
        message: `Found ${links.length} torrent link(s)`
      });
    } catch (error) {
      console.error('Extraction failed:', error);
    }
  }
}

// Function to be injected into page
function extractMagnetLinks() {
  const links = [];

  // Find all magnet links
  document.querySelectorAll('a[href^="magnet:"]').forEach(a => {
    links.push({
      url: a.href,
      text: a.textContent.trim(),
      type: 'magnet'
    });
  });

  // Find .torrent file links
  document.querySelectorAll('a[href$=".torrent"]').forEach(a => {
    links.push({
      url: a.href,
      text: a.textContent.trim(),
      type: 'torrent-file'
    });
  });

  return links;
}

// Listen for tab updates to detect torrent sites
browser.tabs.onUpdated.addListener((tabId, changeInfo, tab) => {
  if (changeInfo.status === 'complete' && tab.url) {
    checkForTorrentSite(tab);
  }
});

async function checkForTorrentSite(tab) {
  // Check if this is a known torrent site
  const torrentSites = [
    'thepiratebay',
```

```
    '1337x',
    'rutracker',
    'nyaa'
  ];

  const isTorrentSite = torrentSites.some(site =>
        tab.url.includes(site));

  if (isTorrentSite) {
    // Update badge to indicate torrent links may be present
    await browser.action.setBadgeText({
      text: '!',
      tabId: tab.id
    });
    await browser.action.setBadgeBackgroundColor({
      color: '#FF6B35',
      tabId: tab.id
    });
  }
}

// Message handling from popup/content scripts
browser.runtime.onMessage.addListener((message, sender,
        sendResponse) => {
  if (message.action === 'getTorrents') {
    return browser.storage.local.get(`torrents_${message.tabId}`)
      .then(data => data[`torrents_${message.tabId}`] || null);
  }

  if (message.action === 'downloadTorrent') {
    return browser.downloads.download({
      url: message.url,
      filename: message.filename || 'torrent-file.torrent'
    });
  }

  if (message.action === 'copyToClipboard') {
    // Use offscreen document for clipboard in Chrome MV3
    return copyToClipboard(message.text);
  }
});

async function copyToClipboard(text) {
  try {
    // For Firefox, we can use the clipboard API directly
    if (typeof navigator.clipboard !== 'undefined' &&
        navigator.clipboard.writeText) {
      await navigator.clipboard.writeText(text);
      return { success: true };
    }
  } catch (error) {
    console.error('Clipboard copy failed:', error);
```

```
    return { success: false, error: error.message };
  }
}
```

11.2 Cross-Browser Content Script

```
// content.js - Content script for torrent extraction
import browser from 'webextension-polyfill';

// Listen for messages from background/popup
browser.runtime.onMessage.addListener((message, sender,
        sendResponse) => {
  if (message.action === 'extract') {
    const data = extractTorrentData();
    return Promise.resolve(data);
  }

  if (message.action === 'highlight') {
    highlightTorrentLinks();
    return Promise.resolve({ success: true });
  }
});

function extractTorrentData() {
  const magnets = [];
  const torrentFiles = [];

  // Extract magnet links
  document.querySelectorAll('a[href^="magnet:"]').forEach(link => {
    magnets.push({
      url: link.href,
      name: link.textContent.trim() || link.title || 'Unknown',
      hash: extractHash(link.href)
    });
  });

  // Extract .torrent file links
  document.querySelectorAll('a[href$=".torrent"]').forEach(link => {
    torrentFiles.push({
      url: link.href,
      name: link.textContent.trim() || 'Unknown torrent'
    });
  });

  return {
    pageUrl: window.location.href,
    pageTitle: document.title,
    magnets,
    torrentFiles,
    extractedAt: new Date().toISOString()
  };
```

```
}

function extractHash(magnetUrl) {
  const match = magnetUrl.match(/xt=urn:btih:([a-zA-Z0-9]+)/i);
  return match ? match[1].toLowerCase() : null;
}

function highlightTorrentLinks() {
  document.querySelectorAll('a[href^="magnet:"]').forEach(link => {
    link.style.backgroundColor = '#FFEB3B';
    link.style.padding = '2px 4px';
    link.style.borderRadius = '3px';
  });
}

// Auto-extract on page load if enabled
async function init() {
  try {
    const { autoExtract } = await
        browser.storage.local.get('autoExtract');
    if (autoExtract) {
      const data = extractTorrentData();
      if (data.magnets.length > 0 || data.torrentFiles.length >
        0) {
        await browser.runtime.sendMessage({
          action: 'autoExtracted',
          data
        });
      }
    }
  } catch (error) {
    console.error('Auto-extract failed:', error);
  }
}

init();
```

11.3 Cross-Browser Popup

```
// popup.js - Extension popup
import browser from 'webextension-polyfill';

async function initPopup() {
  const [tab] = await browser.tabs.query({ active: true,
        currentWindow: true });

  // Get extracted torrents for current tab
  const storageKey = `torrents_${tab.id}`;
  const data = await browser.storage.local.get(storageKey);
  const torrentData = data[storageKey];

  if (torrentData && torrentData.links.length > 0) {
```

```
    renderTorrentList(torrentData.links);
  } else {
    // Trigger extraction
    showLoading();
    try {
      const results = await browser.scripting.executeScript({
        target: { tabId: tab.id },
        func: () => {
          // Inline extraction function
          const links = [];

          document.querySelectorAll('a[href^="magnet:"]').forEach(a
          => {
              links.push({ url: a.href, text: a.textContent.trim(),
          type: 'magnet' });
          });
          return links;
        }
      });

      const links = results[0]?.result || [];
      renderTorrentList(links);
    } catch (error) {
      showError('Failed to extract torrents: ' + error.message);
    }
  }
}

function renderTorrentList(links) {
  const container = document.getElementById('torrent-list');
  container.innerHTML = '';

  if (links.length === 0) {
    container.innerHTML =
        '<p class="empty">No torrent links found on this page.</
        p>';
    return;
  }

  const header = document.createElement('div');
  header.className = 'header';
  header.textContent = `${links.length} torrent link(s) found`;
  container.appendChild(header);

  links.forEach((link, index) => {
    const item = document.createElement('div');
    item.className = 'torrent-item';

    const name = document.createElement('span');
    name.className = 'name';
    name.textContent = link.text || `Torrent ${index + 1}`;
    name.title = link.url;
```

```
    const actions = document.createElement('div');
    actions.className = 'actions';

    const copyBtn = document.createElement('button');
    copyBtn.textContent = 'Copy';
    copyBtn.onclick = () => copyToClipboard(link.url);

    const downloadBtn = document.createElement('button');
    downloadBtn.textContent = 'DL';
    downloadBtn.onclick = () => downloadTorrent(link.url);

    actions.appendChild(copyBtn);
    actions.appendChild(downloadBtn);

    item.appendChild(name);
    item.appendChild(actions);
    container.appendChild(item);
  });
}

async function copyToClipboard(text) {
  try {
    await navigator.clipboard.writeText(text);
    showToast('Copied to clipboard!');
  } catch (error) {
    // Fallback for browsers without clipboard API
    await browser.runtime.sendMessage({ action:
        'copyToClipboard', text });
    showToast('Copied!');
  }
}

async function downloadTorrent(url) {
  try {
    await browser.downloads.download({ url, saveAs: true });
    showToast('Download started!');
  } catch (error) {
    showError('Download failed: ' + error.message);
  }
}

function showLoading() {
  document.getElementById('torrent-list').innerHTML = '<p
        class="loading">Extracting...</p>';
}

function showError(message) {
  document.getElementById('torrent-list').innerHTML = `<p
        class="error">${message}</p>`;
}
```

```
function showToast(message) {
  const toast = document.createElement('div');
  toast.className = 'toast';
  toast.textContent = message;
  document.body.appendChild(toast);
  setTimeout(() => toast.remove(), 2000);
}

// Initialize
document.addEventListener('DOMContentLoaded', initPopup);
```

11.4 Build Configuration with Vite + WXT

```
// wxt.config.ts
import { defineConfig } from 'wxt';

export default defineConfig({
  srcDir: 'src',
  outDir: 'dist',
  manifest: {
    name: '__MSG_extName__',
    description: '__MSG_extDescription__',
    default_locale: 'en',
    permissions: [
      'activeTab',
      'storage',
      'downloads',
      'contextMenus',
      'notifications',
      'scripting',
      'tabs'
    ],
    host_permissions: [
      'http://*/',
      'https://*/'
    ],
    content_scripts: [
      {
        matches: ['<all_urls>'],
        js: ['content-scripts/index.ts'],
        run_at: 'document_idle'
      }
    ],
    web_accessible_resources: [
      {
        resources: ['icons/*', 'assets/*'],
        matches: ['<all_urls>']
      }
    ]
  },
```

```
  // WXT auto-generates per-browser manifests
});

// package.json
{
  "name": "torrent-extractor",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "wxt",
    "dev:firefox": "wxt --browser firefox",
    "build": "wxt build",
    "build:chrome": "wxt build --browser chrome",
    "build:firefox": "wxt build --browser firefox",
    "build:edge": "wxt build --browser edge",
    "build:opera": "wxt build --browser opera",
    "build:all": "wxt build --browser chrome && wxt build --
        browser firefox && wxt build --browser edge && wxt build
        --browser opera",
    "zip": "wxt zip",
    "zip:all": "wxt zip --browser chrome && wxt zip --browser
        firefox && wxt zip --browser edge && wxt zip --browser
        opera",
    "lint": "wxt lint"
  },
  "dependencies": {
    "webextension-polyfill": "^0.12.0"
  },
  "devDependencies": {
    "@types/webextension-polyfill": "^0.12.0",
    "typescript": "^5.6.0",
    "wxt": "^0.20.0"
  }
}
```

---

12. References

Official Documentation

Resource	URL	Description
Chrome Extension API Reference	https://developer.chrome.com/docs/extensions/reference/api	Official Chrome extension APIs
MDN WebExtensions	https://developer.mozilla.org/en-US/docs/Mozilla/	Firefox extension documentation

Resource	URL	Description
	Add-ons/WebExtensions	
Opera Extensions Documentation	https://help.opera.com/en/extensions/	Opera extension docs
Edge Extension Documentation	https://learn.microsoft.com/en-us/microsoft-edge/extensions/	Microsoft Edge extension docs
Yandex Browser Help	https://browser.yandex.ru/help/en/personalization/extension	Yandex extension installation

Tools and Libraries

Resource	URL	Description
webextension-polyfill	https://github.com/mozilla/webextension-polyfill	Mozilla’s official API polyfill
@types/webextension-polyfill	https://www.npmjs.com/package/@types/webextension-polyfill	TypeScript types for polyfill
WXT Framework	https://wxt.dev / https://github.com/wxt-dev/wxt	Next-gen Web Extension framework
vite-plugin-web-extension	https://vite-plugin-web-extension.aklinker1.io	Vite plugin for extensions
web-ext CLI	https://github.com/mozilla/web-ext	Firefox build/sign/publish tool
webext-dynamic-content-scripts	https://github.com/fregante/webext-dynamic-content-scripts	Dynamic content script injection

Store Developer Portals

Store	URL	Notes
Chrome Web Store Dev Console	https://chrome.google.com/webstore/devconsole/	$5 one-time fee

Store	URL	Notes
Firefox AMO Developer Hub	https://addons.mozilla.org/developers/	Free
Opera Add-ons Developer	https://addons.opera.com/developer/	Free
Edge Add-ons Developer	https://partner.microsoft.com/en-us/dashboard/microsoftedge/	Free

Key Research Sources

1. **Mozilla Add-ons Blog** - https://blog.mozilla.org/addons/ - Official Firefox extension news and updates
2. **Chrome Developers Blog** - https://developer.chrome.com/blog/ - Official Chrome extension updates
3. **W3C WebExtensions Community Group** - https://github.com/w3c/webextensions - Cross-browser standardization
4. **MDN Cross-Browser Extension Guide** - https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Build_a_cross_browser_extension
5. **Extension Workshop** - https://extensionworkshop.com/ - Mozilla's official extension development resource

---

Appendix A: Quick Reference Card

A.1 Manifest V3 Minimum Viable Template

```
{
  "manifest_version": 3,
  "name": "Extension Name",
  "version": "1.0.0",
  "description": "Extension description",
  "permissions": ["storage", "activeTab"],
  "action": {
    "default_popup": "popup.html"
  },
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [{
    "matches": ["<all_urls>"],
    "js": ["content.js"]
  }],
```

```
  "icons": {
    "16": "icon16.png",
    "48": "icon48.png",
    "128": "icon128.png"
  }
}
```

A.2 Per-Browser Build Command Cheat Sheet

Target	Command	Output
Chrome (dev)	`wxt --browser chrome`	`.output/chrome-mv3-dev/`
Chrome (prod)	`wxt build --browser chrome`	`.output/chrome-mv3-prod/`
Firefox (dev)	`wxt --browser firefox`	`.output/firefox-mv2-dev/`
Firefox (prod)	`wxt build --browser firefox`	`.output/firefox-mv2-prod/`
Edge (prod)	`wxt build --browser edge`	`.output/edge-mv3-prod/`
Opera (prod)	`wxt build --browser opera`	`.output/opera-mv3-prod/`
All (zip)	`wxt zip --browser all`	`.output/*.zip`

A.3 Permission Mapping

Permission	Chrome	Firefox	Opera	Edge	Use Case
`activeTab`	Yes	Yes	Yes	Yes	Access current tab
`tabs`	Yes	Yes	Yes	Yes	Query all tabs
`storage`	Yes	Yes	Yes	Yes	Local data storage
`downloads`	Yes	Yes	Yes	Yes	Download .torrent files
`contextMenus`	Yes	Yes*	Yes	Yes	Right-click menus
`notifications`	Yes	Yes	Partial	Yes	User notifications
`scripting`	Yes	Yes	Yes	Yes	Content script injection
`clipboardWrite`	Yes	Yes	Yes	Yes	Copy magnet links
`webRequest`	DNR only	Yes (blocking)	DNR only	DNR only	Network monitoring
`host_permissions`	Yes	Yes	Yes	Yes	Site access

*Firefox uses `menus` API which is a superset of `contextMenus`

A.4 Common Gotchas

Issue	Solution
Firefox MV3 requires extension ID	Add `browser_specific_settings.gecko.id`
Service workers lose state	Use `chrome.storage` instead of global variables
Chrome MV3 no DOM in background	Use offscreen documents for DOM operations
`browser.*` doesn't exist in Chrome	Use `webextension-polyfill`
Opera doesn't support `storage.sync`	Use `storage.local` only
Firefox `browser.menus` vs Chrome `chrome.contextMenus`	Polyfill maps them automatically
`tabs.executeScript` returns Promise in Firefox, callback in Chrome MV2	Use polyfill for consistent Promise API
Yandex blocks some extensions	Sign with developer key or distribute as unpacked
Chrome Web Store requires $5 fee	One-time payment per developer account
AMO requires source code for minified builds	Submit unminified source + build instructions
Version format: Firefox doesn't allow letters	Use `x.y.z` format only
`--load-extension` removed from branded Chrome builds (v137+)	Use `--remote-debugging-pipe` + `Extensions.loadUnpacked`

Document compiled from 20+ authoritative sources including MDN, Chrome Developer docs, Mozilla Add-ons Blog, Opera Help, Microsoft Learn, GitHub repositories, and community resources.

All citations use [^{number}] format referencing source URLs.